laptop → vertical ⇒ same laptop / similar laptop

→ big processor

→ big ram

→ big HDD.

→ maintainence is easy

→ scale up is expensive and after a certain pt, impossible

→ SPOF (single pt. of failure)

8GB → commodity → 60 x times = 512GB

↓

128GB / 256GB / 512GB → prices goes up

↑

cost (8 x 65) << cost (1 x 512)

base MS Pro – 1TB / 512GB ← 2x

⋮

8TB / 16TB → cost

horizontal ⇒ add more devices

↓

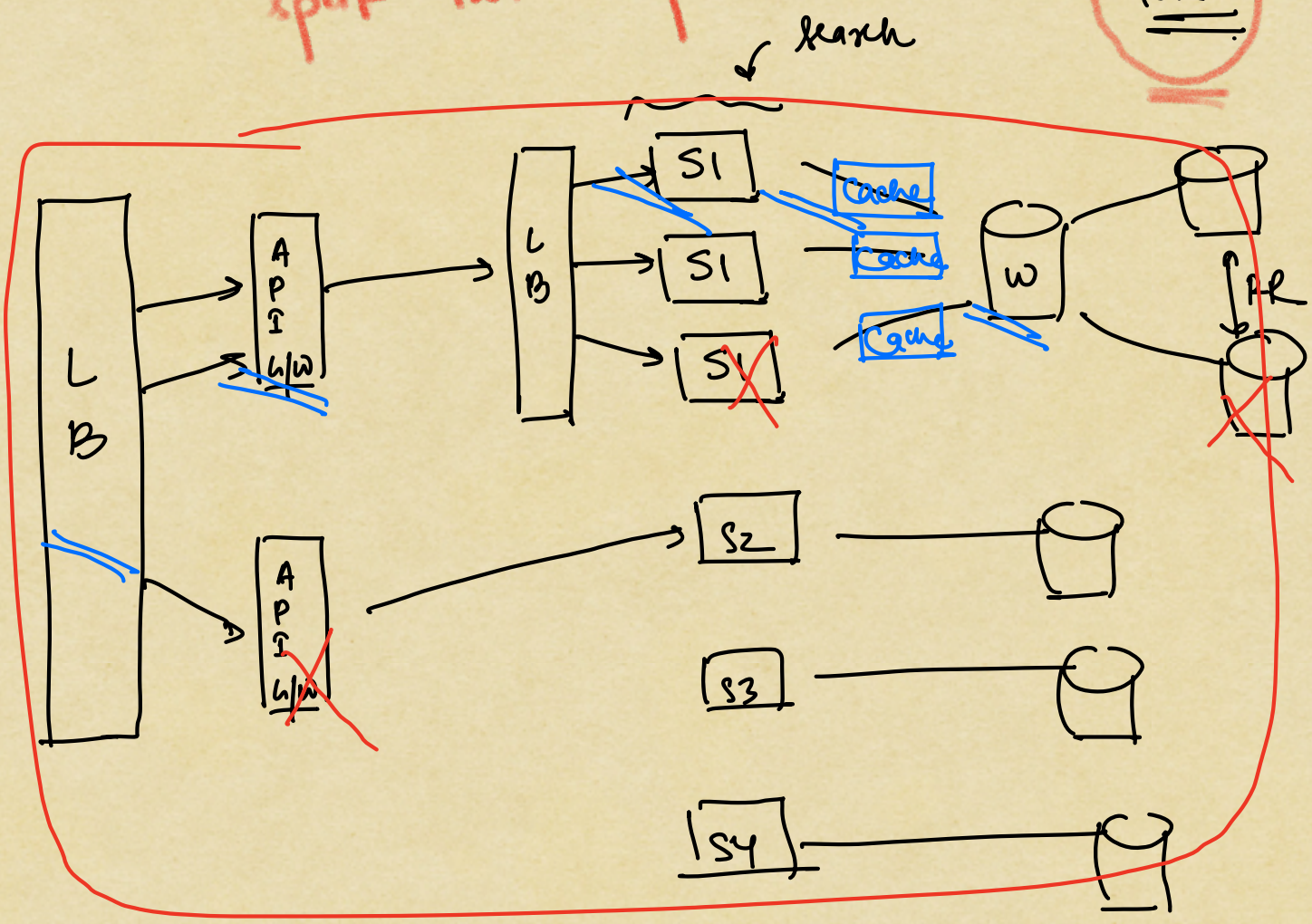
Game ⇒ HTAV ⇒ 80GB → 140GB

Op ⇒ 1 ⇒ 128GB → Pendrive ⇒ 15K

Op 2 ⇒ 10×8GB → pendrive ⇒ 1K ↓ 10K

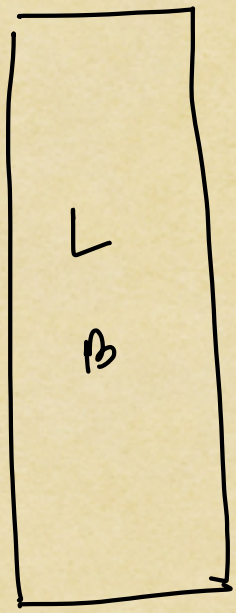
⇒ 15 | 15

split into 10 parts



Load Balancer ⇒ balances the load

→ ~~equal~~ distribution of load



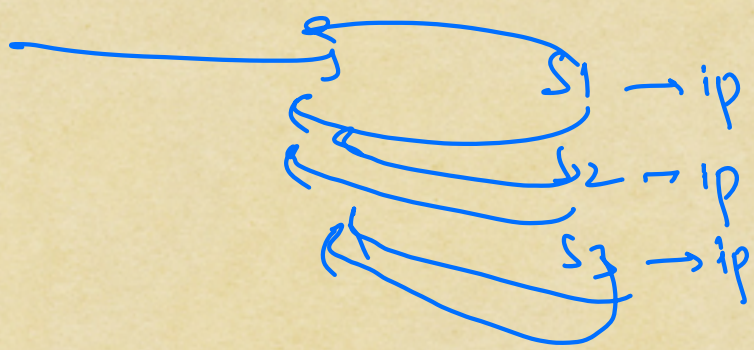
computer that runs load balancing algorithms.

⇒ Can LB be a SPOF?

Ans ⇒ Yes / NO.

→ laptop / server / computer

Round Robin



DNS resolution

universal hashmap.

server

domainName → <ip> ; <list of ip>

⇒ DNS servers (www) → trust / open source

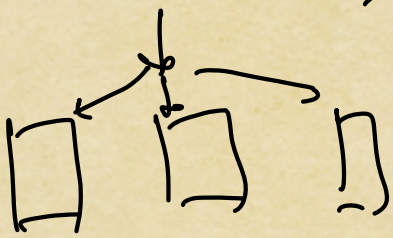
↓ (13)

↓ ICANN.

→ browser cache → ISP DNS → root server

↓

TLD server (.com, .net, ...)



Q. DNS resolution scale?

→ total no.

→ number that actually reaches any TLD server

→ Smart LB → ML algorithms.

→ healthcheck of servers.

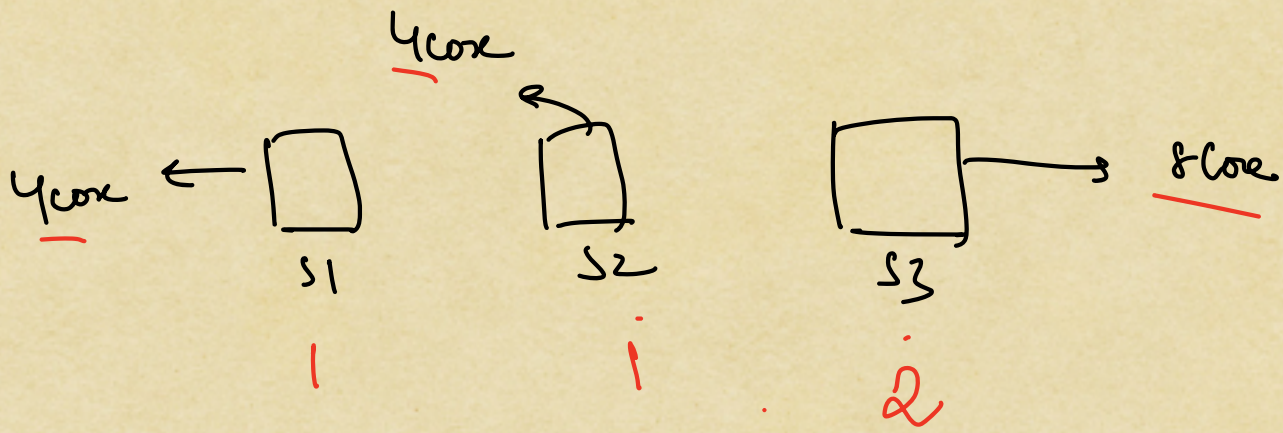
→ response time etc

→ normal LB

→ RR

→ weighted RR (1:1:2 — 1, 2, 3 4 — 5, 6, 7 8)

→ least response time

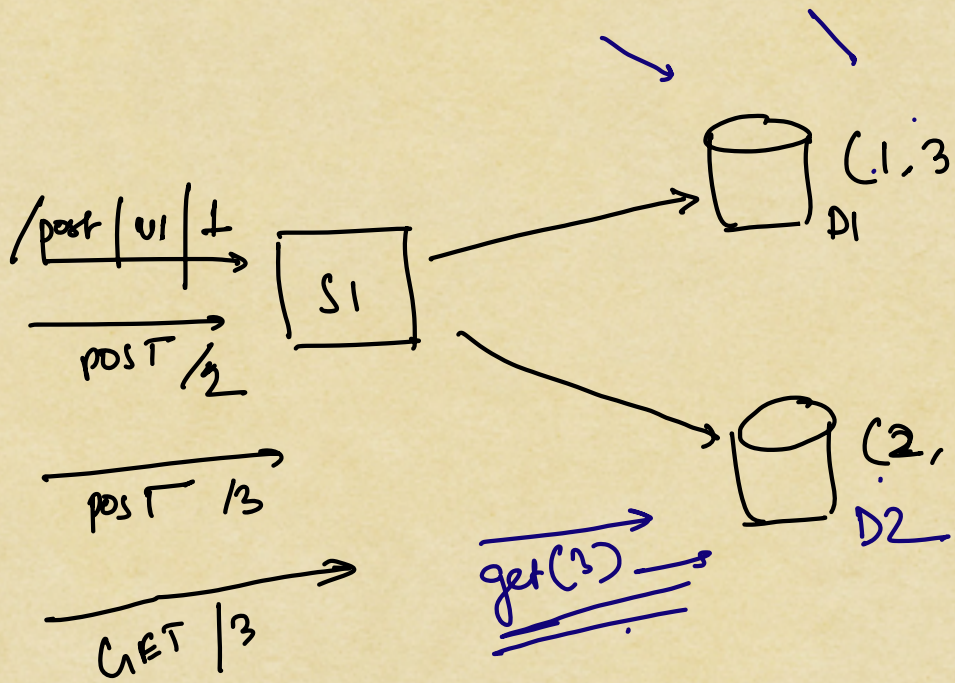


	S1	S2	S3
req 1	✓		
2		✓	
3			✓
4			✓
5	✓		
6		✓	

7

8

⇒ Database Scaling



⇒ hashing

fn(x) ⟶ y

fn(x) ⟶ y

fn(x) ⟶ y

~~fn(y) ⟶ x~~

id = 6 % 4

id = 1 % 4

id = 6 % 4 = 2

Server S1

id = 4 % 4. id = 7 % 4

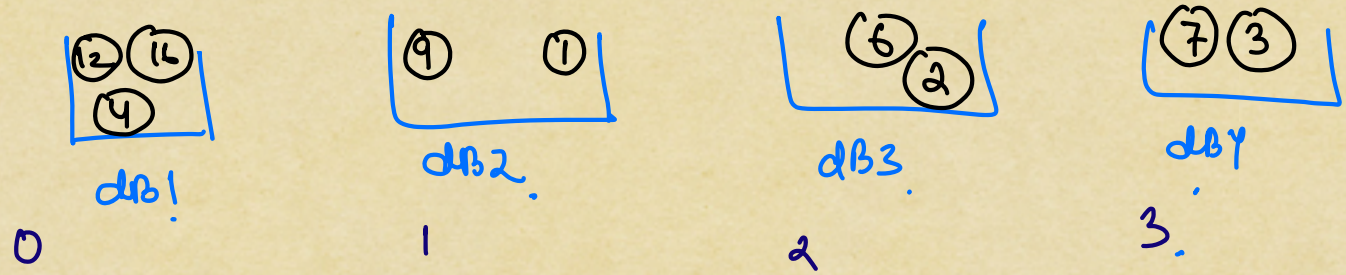
hash(user1) → int % 4

0 - 3

0 - 3 ⇒ equal distribution

→ equal distribution of data into machines

→ we are able to get the exact machine that contains the data.



hashed o/p

⇒ User ⇒

7 ⟶ db4

3 ⟶ db4

9 ⟶ db2

12 ⟶ db1

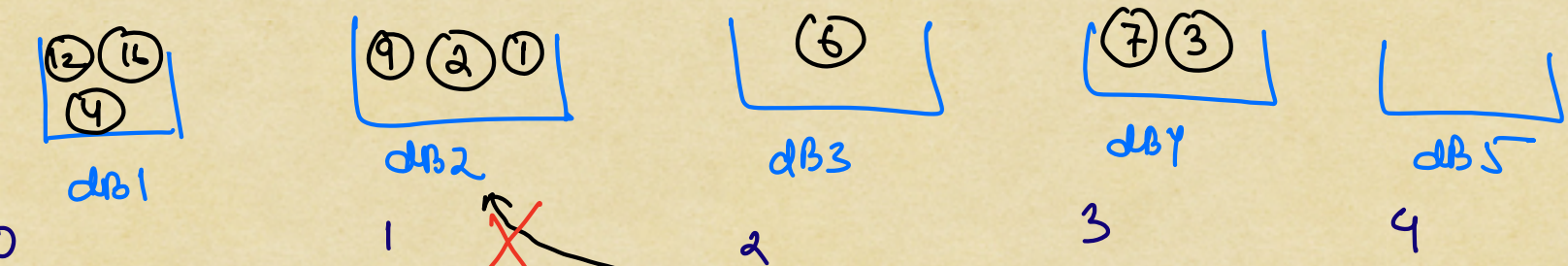
16 ⟶ db1

4 ⟶ db1

2 ⟶ db1

1 ⟶ db2

6 ⟶ db3



16 doesn't exist

16 ~~⟶ db1~~ 16 % 5 ⟶ 1 (db2)

basically we now need to do data redistribution

constantly requires redistribution when adding or removing DBs.

⇒ Consistent hashing

⇒ hash space ⇒ hash(id) % 4

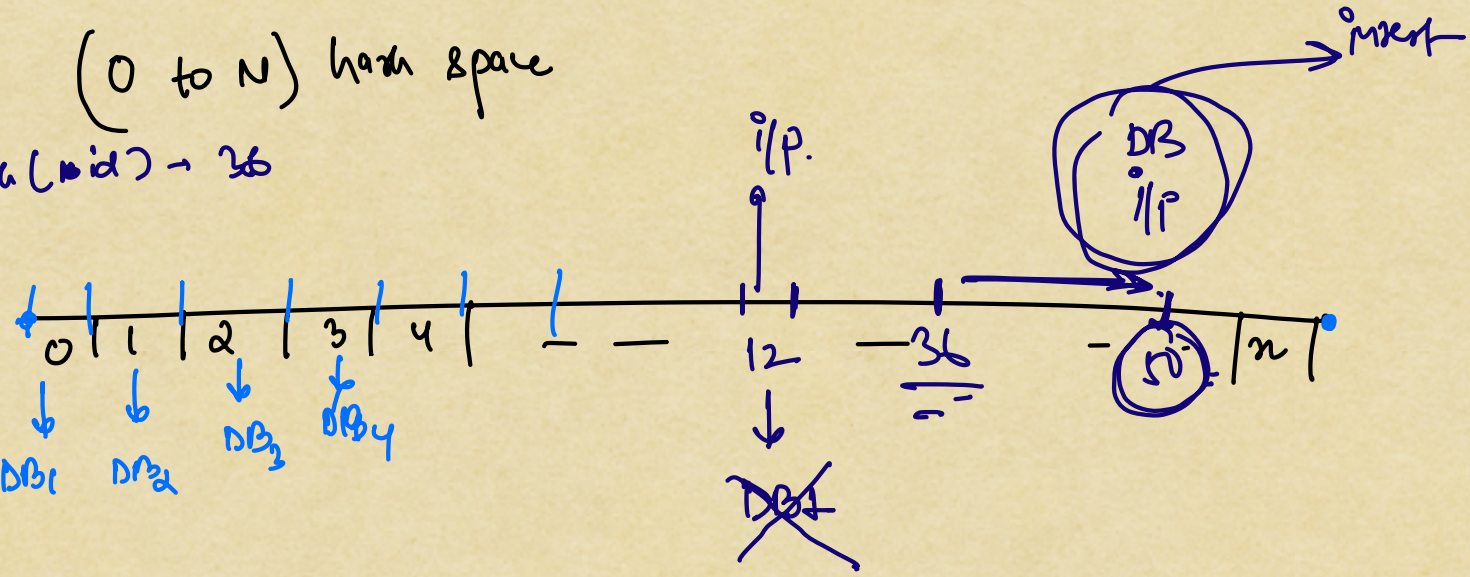
⇒ 0-3

⇒ hash(email) ⇒ (sum of all ascii charac) % 100

⇒ 0-99.

(0 to N) hash space

hash(id) → 36

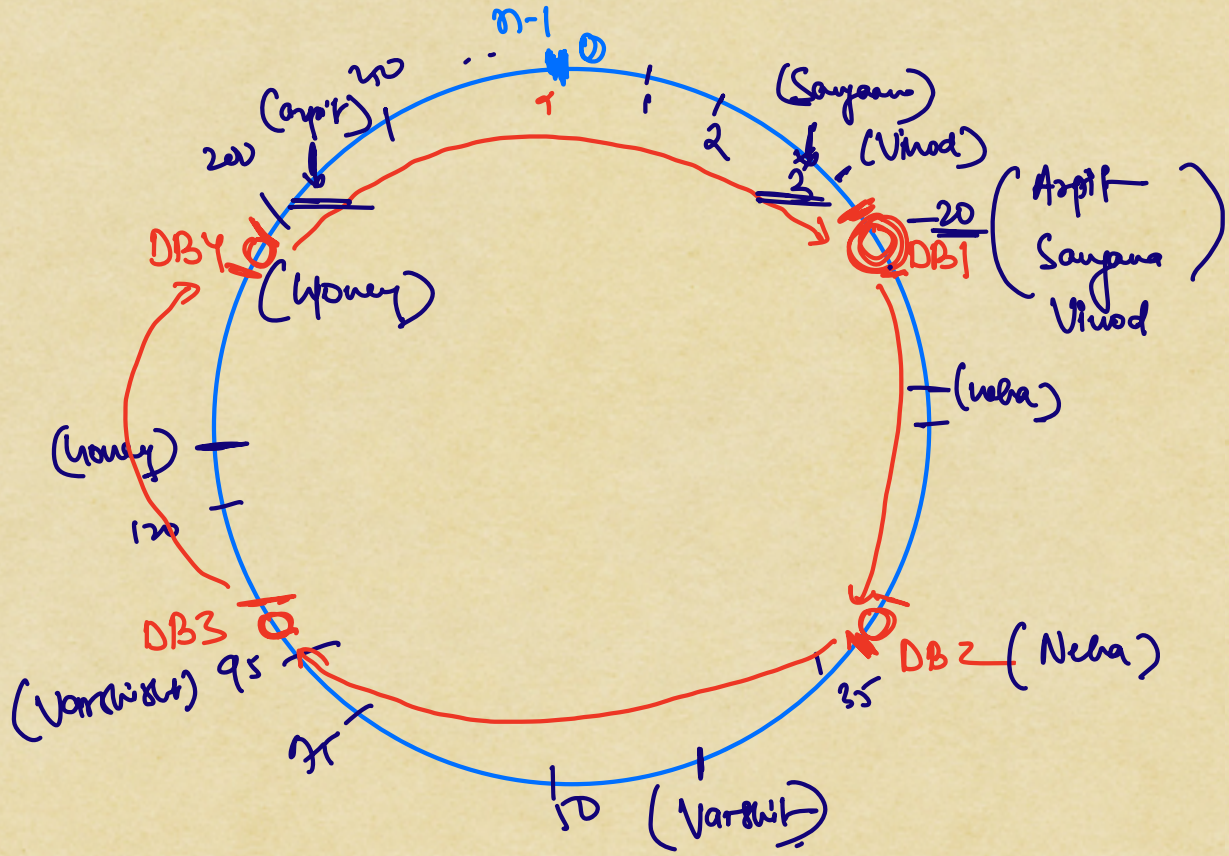


hash(data's identifier)

hash(server's identifier)

↓

IP



hash(sanjana) ⟶ 3

hash(neha) ⟶

hash(arpit) ⟶ 220

hash(varshit..)

hash(Honey) ⟶

hash(DB1 ip)

" (DB2 ip)

" (DB3 ..)

" (DB4 ..)

hash(vinod) ⟶ 12

⟶ 20

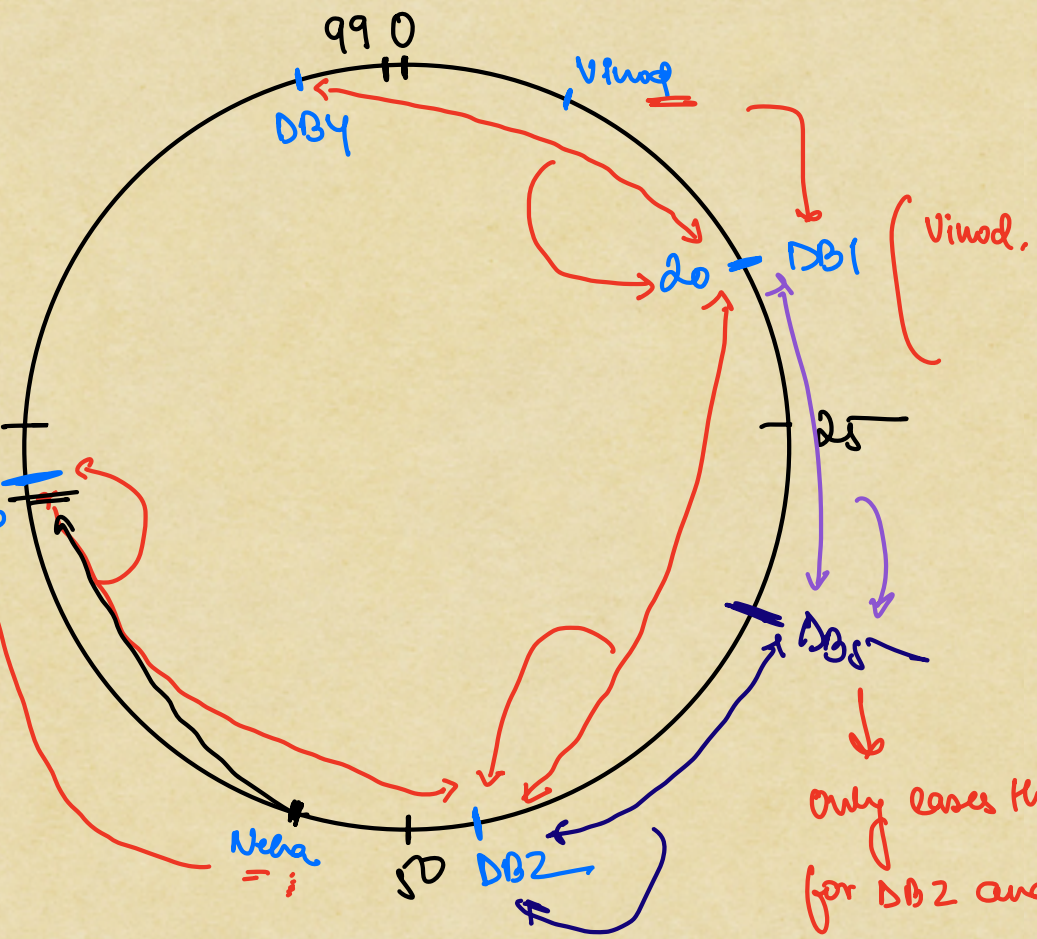
12 ⟶ DB1

DB1 → 20

DB2 → 45

DB3 → 70

DB4 → 95



Nehra

hash(nehra) → 56

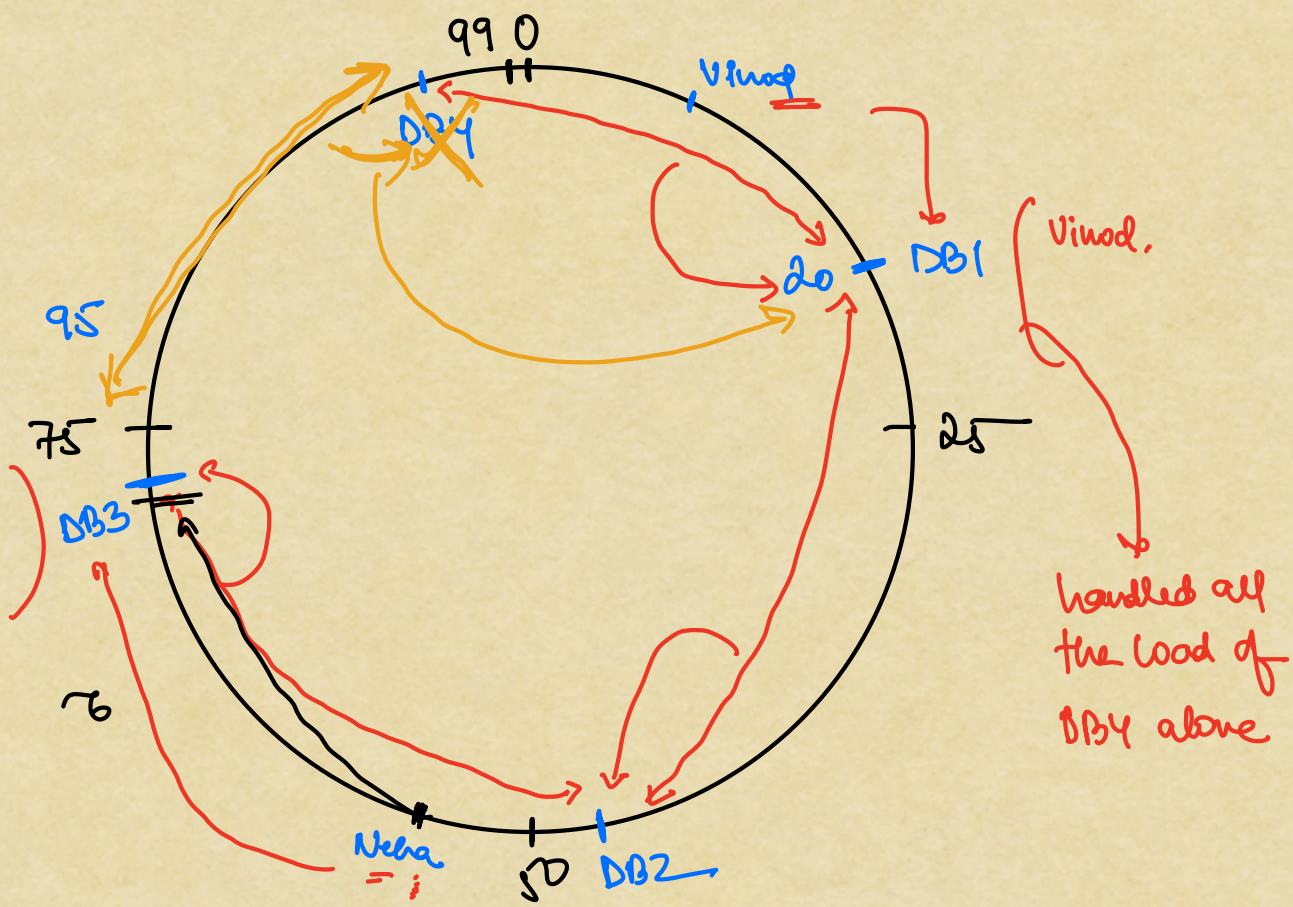
hash(vinod)

↓

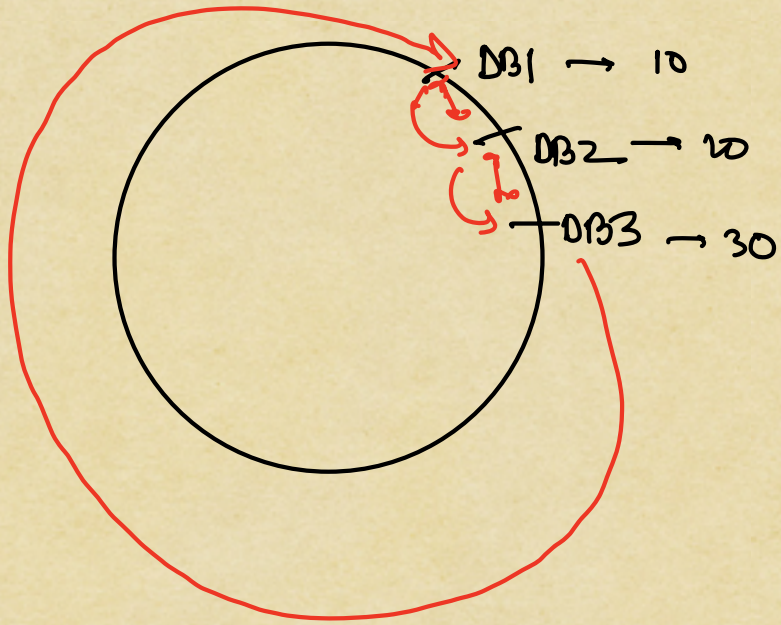
12

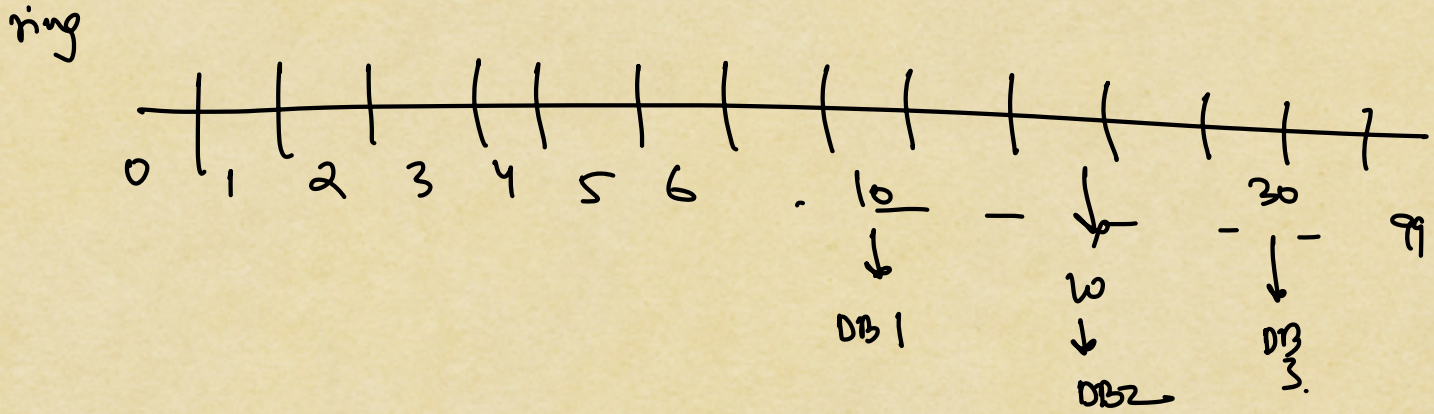
only eases the load for DB2 and not the entire system

xad → Nehra → hash(nehra) ⟹ 50 → DB3



⇒ gaurantee ⇒ dB will distributed evenly

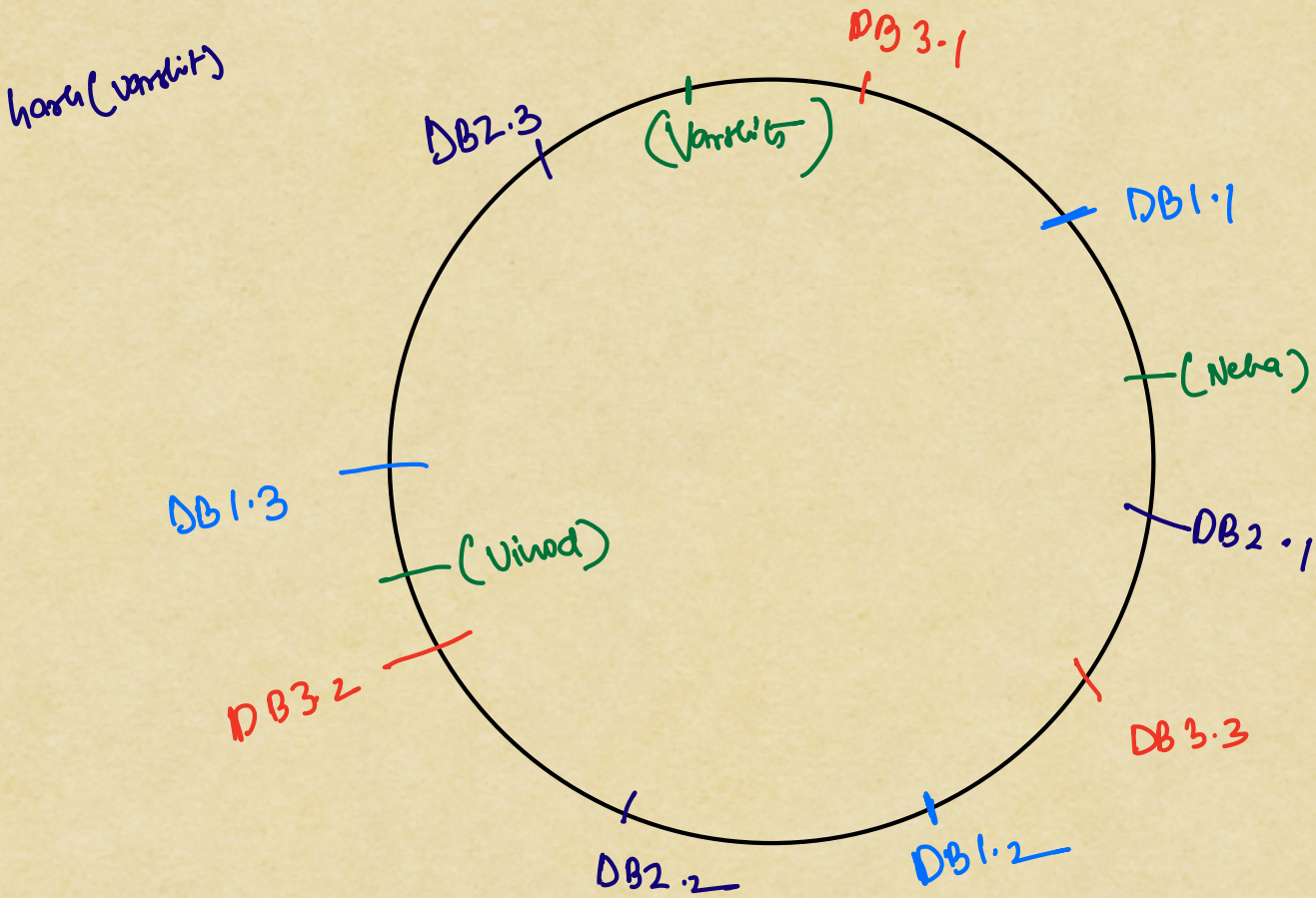




DB1 ⇒ DB1.1, DB1.2, DB1.3 ⟹ DB1 (Vinod,

DB2 ⇒ DB2.1 DB2.2 DB2.3 ⟹ DB2 (Neha,

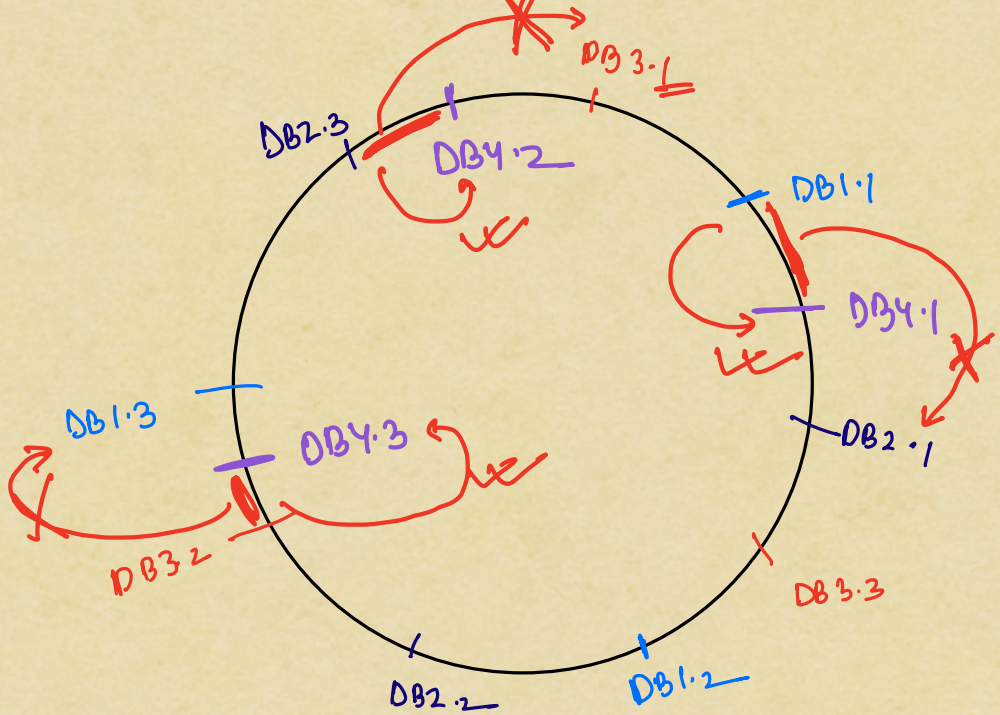
DB3 ⇒ DB 3.1 DB3.2 DB 3.3 ⇒ DB3 (Vamsi)

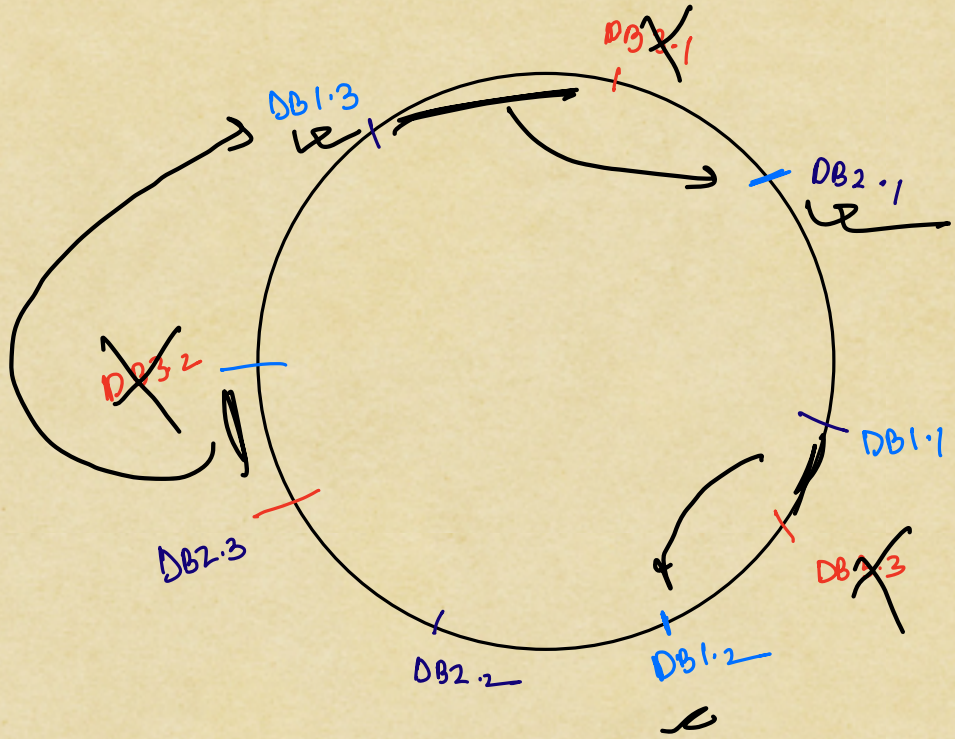


⇒ add a server

⇒ DB4

DB 4.1 | DB 4.2 | DB 4.3



⇒ when a DB dies

DB3 · dies